

2026 School on Electron-Phonon Physics and Many-Body perturbation theory

Machine learning potentials

Hands-on Session (Tue.5)

Hands-on based on MACE (v0.3.16), ASE (v3.26.0), phonopy (v2.45.1)

Introduction

In this tutorial, we will show how to train and use machine learning interatomic potentials (MLIPs, often also called machine learning force fields or MLFFs) to accelerate the calculation of vibrational properties and molecular dynamics simulations. In general, an MLIP is useful whenever a large number of DFT total-energy calculations in supercells is required. Accurate MLIPs can offer near first-principles accuracy at a greatly reduced computational cost. There are a number of freely available implementations of MLIPs: here we use MACE, a very accurate and efficient equivariant message-passing graph tensor network model.¹

Using the [MACE](#) framework, we will train two MLIP models, one for crystalline zirconia (ZrO_2) and one for various phases of silica (SiO_2). To perform molecular dynamics and phonon calculations, we will also use the popular Python libraries [ASE](#) and [phonopy](#). The full documentation can be found at these links:

<https://mace-docs.readthedocs.io/en/latest/guide>

<https://ase-lib.org/ase>

<https://phonopy.github.io/phonopy>

Set-up

We start by creating a python virtual environment called MACE in the work directory (\$WORK), after loading the python3/3.9.2 module:

```
$ module load python3/3.9.2
$ python3 -m venv $WORK/MACE
```

To install all the packages needed, first request a short interactive job using iddev:

```
$ iddev -N 1 -n 1 -m 30 -A DMR23048 --reservation=MATCSSI_Norm_June16
```

Now paste the following commands:

```
$ source $WORK/MACE/bin/activate
$ export CC=gcc; export CXX=g++; unset PYTHONPATH
$ pip install --upgrade pip
$ pip install --only-binary=:all: pandas==2.2.3
$ pip install h5py phonopy mace-torch
```

Note: ASE will be installed automatically when installing MACE.

This should take less than 5 minutes. If all goes well, you should see the following on your terminal (just the last long line is reproduced here):

¹You can check out the theory paper: [I. Batatia, D. P. Kovacs, G. Simm, C. Ortner, and G. Csányi, Adv. Neural Inf. Process. Syst. 35, 11423 \(2022\).](#)

```
Successfully installed GitPython-3.1.50 MarkupSafe-3.0.3 PyYAML-6.0.3 ase-3.26.0
configargparse-1.7.5 contourpy-1.3.0 cycler-0.12.1 e3nn-0.4.4 filelock-3.19.1 fonttools-4.60.2
fsspec-2025.10.0 gitdb-4.0.12 h5py-3.14.0 importlib-resources-6.5.2 jinja2-3.1.6
kiwisolver-1.4.7 lightning-utilities-0.15.2 lmbd-2.2.1 mace-torch-0.3.16
...
torchmetrics-1.8.2 tqdm-4.68.1 triton-3.2.0 typing-extensions-4.15.0 wcwidth-0.8.0 zipp-3.23.1
```

After everything is done, don't forget to exit the interactive job:

```
$ exit
```

► Activate the virtual environment and make sure you have the executable:

```
$ source $WORK/MACE/bin/activate
$ which mace_run_train
```

The output should be (yours will be different, depending on your \$WORK path):

```
/work2/11514/cverdi/frontera/MACE/bin/mace_run_train
```

Note: You should now see (MACE) prepended to your terminal prompt.

Now navigate to your scratch directory, download the tar file for this tutorial, and extract it:

```
$ cd $SCRATCH
$ wget --content-disposition "https://drive.google.com/uc?export=download&id=1SRTbjfswZfuSHEkomZ7xaoZnU0qrDOVN"
$ tar -xvf Tue.5.Verdi.tar; cd Tue.5.Verdi
```

You should see the four subdirectories containing the files required for the four exercises in this tutorial:

```
$ ls

exercise1 exercise2 exercise3 exercise4
```

Exercise 1

In this exercise we will train a MACE potential for ZrO_2 and assess its accuracy on training, validation and test data. At ambient pressure, ZrO_2 has three structural phases: cubic (space group $Fm\bar{3}m$) at high temperatures, tetragonal (space group $P4_2/nmc$) between about 2570 K and 1400 K, and monoclinic (space group $P2_1/c$) below 1400 K. We will perform structural relaxations for all three polymorphs and compute their phonon dispersions using MACE, comparing results with reference DFT calculations.

1.1 MACE training for ZrO_2

Navigate to the exercise directory and inspect the content:

```
$ cd exercise1; ls
```

The focus of this tutorial is not to generate the *ab initio* training data from scratch, as this would take too much time, but rather to train and use the potentials. We therefore use existing datasets. These training data would normally come from DFT calculations on relaxed and distorted supercell structures, molecular dynamics (MD) snapshots, or active-learning workflows.

The folder database/ in exercise1/ contains the training data we will use to train the MACE potential for ZrO_2 , as well as separate test data:

```
$ ls database
```

```
zro2-test.xyz  zro2-train.xyz
```

These data were calculated using the PBEsol exchange-correlation functional in [npj Comput. Mater.](#) **7, 156 (2021)** and contain 592 configurations sampled 'on the fly' from MD simulations at various temperatures spanning all three structural phases of ZrO_2 . They are in extended XYZ (extxyz) format, which is an enhanced version of the basic XYZ format with extra columns for additional per-atom properties (e.g., forces) as well as per-frame parameters (e.g., lattice, energy...). Energies are in eV, forces in eV/Å, and stresses in eV/Å³.

Note: If you want to generate your own dataset with Quantum ESPRESSO, the resulting output can be read using ASE, for example, and converted into extended XYZ format for training.

► Train the potential by submitting the job script `run1.sh`:

```
$ sbatch run1.sh
```

```
--
#!/bin/bash
#SBATCH -J myjob      # Job name
#SBATCH -N 1          # Total # of nodes
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=28 # Threads
#SBATCH -t 00:30:00  # Run time (hh:mm:ss)
#SBATCH -A DMR23048
#SBATCH -p small
#SBATCH --reservation=MATCSSI_Norm_June16

module load python3/3.9.2
source $WORK/MACE/bin/activate
unset PYTHONPATH

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export MKL_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OPENBLAS_NUM_THREADS=$SLURM_CPUS_PER_TASK
export NUMEXPR_NUM_THREADS=$SLURM_CPUS_PER_TASK

mace_run_train \
  --name="zro2_MACE_model" \
  --train_file="database/zro2-train.xyz" \
  --valid_fraction=0.1 \
  --energy_key="energy" \
  --forces_key="forces" \
  --stress_key="stress" \
  --compute_stress=True \
  --loss="huber" \
  --test_file="database/zro2-test.xyz" \
  --E0s="{40: -2.07574957, 8: -1.79537657}" \
  --model="ScaleShiftMACE" \
  --num_interactions=2 \
  --num_channels=64 \
  --correlation=3 \
  --max_L=0 \
  --r_max=4 \
  --batch_size=10 \
  --max_num_epochs=15 \
  --ema \
  --ema_decay=0.99 \
  --amsgrad \
  --default_dtype="float32" \
  --device=cpu \
  --seed=123 \
  --swa
```

This will take about 7 minutes to complete using 28 CPU threads. In the meantime, let's inspect and understand the job script. After loading the MACE python environment, it uses the executable `mace_run_train` to train a MACE model. The most important options are noted below:

- The training file is specified with the `--train_file` flag. The validation set to monitor the loss during training can either be specified as a separate file using `--valid_file`, or as a fraction of the training set using `--valid_fraction` (as done here: 10% of the data). The optional test set, which is evaluated only at the end of the training, can be specified using `--test_file`.
- When parsing the data files, energies, forces and stresses are read using the keywords specified with `--energy_key`, `--forces_key`, `--stress_key` (you can inspect the `.xyz` file to check they are correct).
- We ask the model to compute stresses, not just energies and forces, and to include them in the loss (using a so-called Huber loss function). The energy, force and stress weights in the loss are kept to the default values but can also be explicitly set via `--energy_weight`, `--forces_weight`, `--stress_weight`. Isolated atomic energies are specified by the `--E0s` keyword; if not available, MACE can estimate them using `--E0s="average"`.
- The model uses 2 message passing layers (`--num_interactions`) as recommended, 64 invariant messages (`--num_channels` sets the number of messages, while `--max_L` controls their symmetry), correlation order for the many-body expansion of the on-site energy of 3 (`--correlation`) and cutoff radius of 4 to create the local environment in each layer (`--r_max`).
- The model is trained for 15 epochs; during the last few epochs, the weight on the energy error is increased, following the recommended strategy (this is controlled by `--swa` or `--stage_two`).

This is a small model, with few training epochs, for illustration purposes, but it is already very good as we will see. For improved accuracy, but increased computational cost, one can increase `--max_L` to 1 (medium) or 2 (high), and the model size (`--num_channels`) to 128 or 256. We are also using a reduced precision (`float32`).

Note: All the available options, including the default parameters, can be found by typing

```
$ mace_run_train --help
```

Note: Faster performance can be achieved using GPUs. In that case, set `--device=cuda` instead of `cpu` and request a GPU node.

Note: Once the model settings have been chosen, one can also retrain a final model using the full dataset, i.e. by setting the validation fraction to zero.

Note: Atoms separated by more than `r_max` do not directly communicate, but can still communicate by proxy if there are multiple message-passing layers. The effective receptive field of the model is thus `num_interactions * r_max`.

You can now look at the logfile written in the `logs/` folder that has been created by the program:

```
$ cat logs/zro2_MACE_model_run-123.log
```

You should see something similar to this (only some of the output is reproduced here):

```
INFO: =====VERIFYING SETTINGS=====
INFO: MACE version: 0.3.16
INFO: Using CPU
INFO: =====LOADING INPUT DATA=====
INFO: Using heads: ['Default']
INFO: Using the key specifications to parse data:
...
...
INFO: Total Training set [energy: 592, stress: 592, virials: 0, dipole components: 0, head: 592, elec_temp: 0, ...
INFO: No validation set provided, splitting training data instead.
INFO: Using random 10% of training set for validation with indices saved in: ./zro2_MACE_model_valid_indices_123.txt
...
...
```

```

INFO: Total Test set [energy: 200, stress: 200, virials: 0, dipole components: 0, head: 200, elec_temp: 0, ...
INFO: Total number of configurations: train=533, valid=59, tests=[Default_Default: 200],
...
...
INFO: =====MODEL DETAILS=====
INFO: Building model
INFO: Message passing with 64 channels and max_L=0 (64x0e)
INFO: 2 layers, each with correlation order: 3 (body order: 4) and spherical harmonics up to: l=3
INFO: 8 radial and 5 basis functions
INFO: Radial cutoff: 4.0 A (total receptive field for each atom: 8.0 A)
INFO: Distance transform for radial basis functions: None
INFO: Hidden irreps: 64x0e
...
...
INFO: =====TRAINING=====
INFO: Started training, reporting errors on validation set
INFO: Loss metrics on validation set
INFO: Initial: head: Default, loss=0.12013891, RMSE_E_per_atom= 1115.37 meV, RMSE_F= 1554.26 meV / A, RMSE_stress= ...
INFO: Epoch 0: head: Default, loss=0.03359633, RMSE_E_per_atom= 176.05 meV, RMSE_F= 473.67 meV / A, RMSE_stress= ...
INFO: Epoch 1: head: Default, loss=0.02060085, RMSE_E_per_atom= 11.33 meV, RMSE_F= 296.16 meV / A, RMSE_stress= ...
INFO: Epoch 2: head: Default, loss=0.01419945, RMSE_E_per_atom= 12.69 meV, RMSE_F= 206.61 meV / A, RMSE_stress= ...
INFO: Epoch 3: head: Default, loss=0.01142215, RMSE_E_per_atom= 7.51 meV, RMSE_F= 167.15 meV / A, RMSE_stress= ...
INFO: Epoch 4: head: Default, loss=0.00973036, RMSE_E_per_atom= 5.27 meV, RMSE_F= 142.32 meV / A, RMSE_stress= ...
INFO: Epoch 5: head: Default, loss=0.00881809, RMSE_E_per_atom= 5.15 meV, RMSE_F= 129.80 meV / A, RMSE_stress= ...
INFO: Epoch 6: head: Default, loss=0.00811683, RMSE_E_per_atom= 5.37 meV, RMSE_F= 119.54 meV / A, RMSE_stress= ...
INFO: Epoch 7: head: Default, loss=0.00766044, RMSE_E_per_atom= 4.39 meV, RMSE_F= 113.13 meV / A, RMSE_stress= ...
INFO: Epoch 8: head: Default, loss=0.00728720, RMSE_E_per_atom= 4.10 meV, RMSE_F= 108.02 meV / A, RMSE_stress= ...
INFO: Changing loss based on Stage Two Weights
INFO: Epoch 9: head: Default, loss=0.13930638, RMSE_E_per_atom= 14.18 meV, RMSE_F= 108.14 meV / A, RMSE_stress= ...
INFO: Epoch 10: head: Default, loss=0.11623987, RMSE_E_per_atom= 5.60 meV, RMSE_F= 105.47 meV / A, RMSE_stress= ...
INFO: Epoch 11: head: Default, loss=0.10921609, RMSE_E_per_atom= 2.35 meV, RMSE_F= 103.40 meV / A, RMSE_stress= ...
INFO: Epoch 12: head: Default, loss=0.10692976, RMSE_E_per_atom= 2.25 meV, RMSE_F= 102.33 meV / A, RMSE_stress= ...
INFO: Epoch 13: head: Default, loss=0.10499918, RMSE_E_per_atom= 1.98 meV, RMSE_F= 101.45 meV / A, RMSE_stress= ...
INFO: Epoch 14: head: Default, loss=0.10369387, RMSE_E_per_atom= 1.95 meV, RMSE_F= 100.82 meV / A, RMSE_stress= ...
INFO: Training complete
INFO:
INFO: =====RESULTS=====
...
...
INFO: Error-table on TRAIN and VALID:
+-----+-----+-----+-----+-----+
| config_type | RMSE E / meV / atom | RMSE F / meV / A | relative F RMSE % | RMSE Stress (Virials) / meV / A (A^3) |
+-----+-----+-----+-----+-----+
| train_Default | 2.2 | 94.8 | 6.49 | 2.4 |
| valid_Default | 2.0 | 100.8 | 6.52 | 2.1 |
+-----+-----+-----+-----+-----+
INFO: Evaluating Default_Default ...
INFO: Error-table on TEST:
+-----+-----+-----+-----+-----+
| config_type | RMSE E / meV / atom | RMSE F / meV / A | relative F RMSE % | RMSE Stress (Virials) / meV / A (A^3) |
+-----+-----+-----+-----+-----+
| Default_Default | 1.8 | 93.6 | 6.35 | 2.5 |
+-----+-----+-----+-----+-----+
INFO: Done

```

Note: Your final errors might be slightly different, especially for energies, due to the limited size of the training dataset.

We will use the ‘stage two’ model produced at the end, with changed weights of energies, forces and stresses as explained above. It is written in the file `zro2_MACE_model_stagetwo.model`: we need this file for subsequent calculations.

► Observe how the loss and root-mean square errors (RMSEs) are decreasing during training. What are the final training, validation and test errors? How do they compare?

► Use the `mace_eval_configs` script to evaluate the trained model on both the train and test datasets and plot the predicted vs. reference DFT data. First, submit this job:

```
$ sbatch run2.sh
```

```

--
#!/bin/bash
#SBATCH -J myjob
#SBATCH -N 1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=28
#SBATCH -t 00:30:00
#SBATCH -A DMR23048
#SBATCH -p small
#SBATCH --reservation=MATCSSI_Norm_June16

module load python3/3.9.2
source $WORK/MACE/bin/activate
unset PYTHONPATH

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export MKL_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OPENBLAS_NUM_THREADS=$SLURM_CPUS_PER_TASK
export NUMEXPR_NUM_THREADS=$SLURM_CPUS_PER_TASK

mace_eval_configs \
  --configs database/zro2-test.xyz \
  --model zro2_MACE_model_stagetwo.model \
  --output test_mace_eval.xyz \
  --device=cpu \
  --compute_stress \
  --default_dtype="float32"

mace_eval_configs \
  --configs database/zro2-train.xyz \
  --model zro2_MACE_model_stagetwo.model \
  --output train_mace_eval.xyz \
  --device=cpu \
  --compute_stress \
  --default_dtype="float32"

```

This should be quick; once done, execute the python script `plot-parity.py` that is already in the folder (reported in Appendix 1):

```
$ python plot-parity.py
```

This should produce the figure `plot-parity.png`, which you can visualise using:

```
$ eog plot-parity.png
```

either from Frontera if you logged in with `ssh -X` (slow), or downloading it to your computer first. It should look like the figure in the following page.

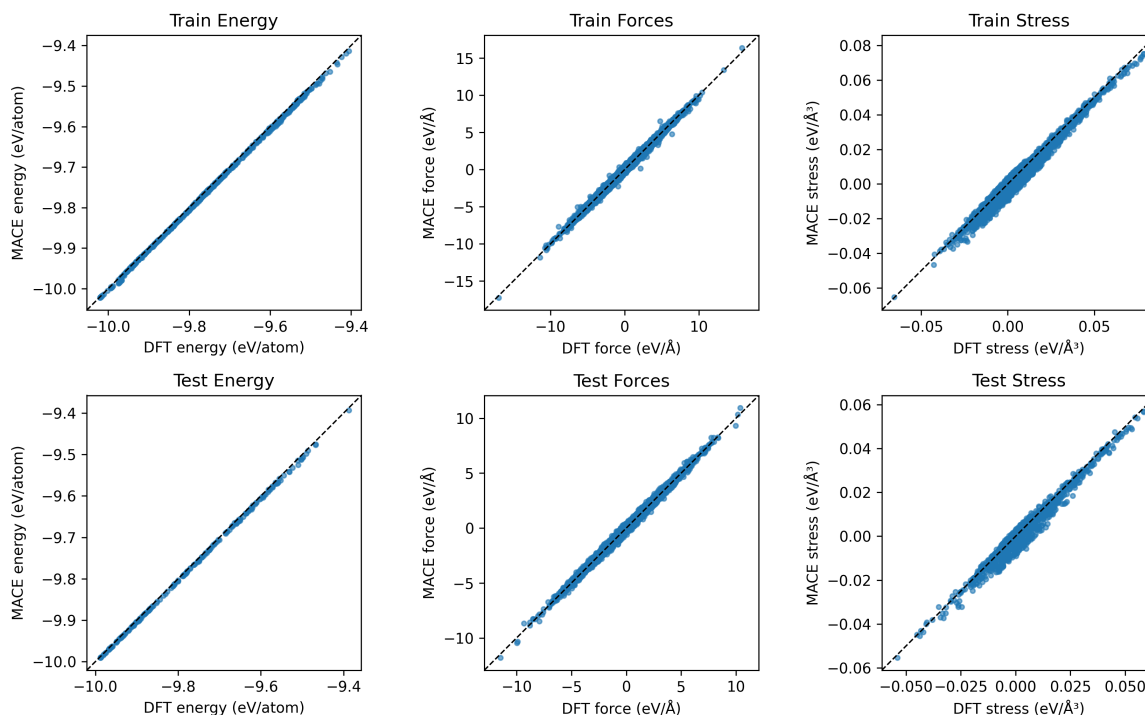
Note: `mace_run_train` also produced the plot `zro2_MACE_model_run-123_train.Default_stage.two.png` in the folder `results/`. Take a look at it: it is similar to what we have seen, so you can now generate similar custom plots.

► At home, try changing the weight of energies, forces and stresses, the model size, and the number of epochs.

A slightly larger, more accurate model was precomputed prior to the tutorial and is provided in the folder `medium/`:

```
$ ls medium/*MACE*
```

```
zro2_MACE_model_stagetwo_medium.model  zro2_MACE_model_run_medium.log
```



► Verify the details of this ‘medium’ model by inspecting some of the INFO printed in the .log file. Identify the key parameters changed so that you may re-train the model yourself if you wish. How much do the RMSEs improve?

► Try using `mace_eval_configs` again to evaluate this model on the training and test datasets, and produce a similar plot as above. For reference, modified `run2.sh` and `plot-parity.py` scripts are provided in the folder `medium/`, as well as a sample resulting plot. Run these scripts in the folder `medium/`.

1.2 Phonon dispersions

What can we do with the MACE model now? We can use it through the [ASE](#) calculator in order to perform many types of calculation, such as geometry optimizations or MD simulations.

► First, relax the unit cells of cubic, tetragonal and monoclinic ZrO_2 . Initial structures are provided in the folder `structures/`:

```
$ ls structures
```

```
zro2-c.cif  zro2-m.cif  zro2-t.cif
```

Note: These are conventional 12-atom unit cells for each structure. The tetragonal one is obtained with a $\sqrt{2} \times \sqrt{2} \times 1$ rotation of the 6-atom unit cell.

```
$ sbatch run3.sh
```

The first part of the job script (up to the `export` commands) is as usual, but now we run the python script `relax.py`:

```
--
...
run3.sh
```

```
...
sed -i "s/torch\.set_num_threads\.*/torch.set_num_threads(${SLURM_CPUS_PER_TASK})/" relax.py
python relax.py zro2-c
python relax.py zro2-t
python relax.py zro2-m
```

```
# relax.py
import sys
import torch
from ase.io import read, write
from ase.optimize import BFGS
from ase.filters import FrechetCellFilter
from mace.calculators import MACECalculator

torch.set_num_threads(8)
name = sys.argv[1]
atoms = read(f"structures/{name}.cif")
atoms.calc = MACECalculator( model_paths="zro2_MACE_model_stagetwo.model",
                             device="cpu", default_dtype="float32" )

# Relax atoms + cell
filt = FrechetCellFilter(atoms)
opt = BFGS(filt, trajectory=f"{name}-relax.traj", logfile=f"{name}-relax.log")
opt.run(fmax=0.01)

write(f"{name}-relax.cif", atoms)
```

Inspect how many iterations were performed for each structure and their energies from the files `name-relax.log`, and the final structures obtained in `name-relax.cif` (where `name` is `zro2-c`, `zro2-t`, `zro2-m`).

► Try filling the table below to compare the model results (MACE small) with the reference DFT values.² How well does the model reproduce the equilibrium geometries? You can also perform the same calculations using the provided ‘medium’ model and compare.

	PBEsol	MACE small	MACE medium
Cubic			
a (Å)	5.069		
$E_c - E_t$ (eV)	0.230		
Tetragonal			
a (Å)	5.074		
c (Å)	5.173		
$E_t - E_m$ (eV)	0.317		
Monoclinic			
a (Å)	5.133		
b (Å)	5.215		
c (Å)	5.300		
β (deg)	99.56		

► Calculate the phonon dispersions for each structure using finite displacements. To do this, we run the python script `phonons.py` with `run4.sh`, which executes:

```
-- run4.sh
...
...
sed -i "s/torch\.set_num_threads\.*/torch.set_num_threads(${SLURM_CPUS_PER_TASK})/" phonons.py
for name in zro2-c zro2-t zro2-m; do
    python phonons.py $name
```

²From [npj Comput. Mater.](#) **7**, 156 (2021).

```
phonopy-bandplot --gnuplot ${name}-band.yaml > ${name}-band.dat
done
```

```
$ sbatch run4.sh
```

```
#                                                                 phonons.py
import sys
import torch
import numpy as np
from ase import Atoms
from ase.io import read
from mace.calculators import MACECalculator
from phonopy import Phonopy
from phonopy.structure.atoms import PhonopyAtoms
from phonopy.file_IO import write_FORCE_CONSTANTS, parse_BORN
from phonopy.phonon.band_structure import get_band_qpoints_and_path_connections
from phonopy.units import Hartree, Bohr

torch.set_num_threads(8)
name = sys.argv[1]
atoms = read(f"{name}-relax.cif")
sc_size = [2, 2, 2]
calc = MACECalculator( model_paths="zro2_MACE_model_stagetwo.model",
                      device="cpu", default_dtype="float32" )

unitcell = PhonopyAtoms( symbols=atoms.get_chemical_symbols(),
                        cell=atoms.cell.array, scaled_positions=atoms.get_scaled_positions() )
# Choose supercell size
phonon = Phonopy( unitcell, supercell_matrix=np.diag(sc_size), primitive_matrix="auto", symprec=1e-4)
# Generate finite displacements (in Å)
phonon.generate_displacements(distance=0.015)
supercells = phonon.supercells_with_displacements
print(f"Number of displaced supercells: {len(supercells)}")

forces = []
for i, scell in enumerate(supercells):
    scell ase = Atoms( symbols=scell.symbols,
                    cell=scell.cell, scaled_positions=scell.scaled_positions, pbc=True )
    scell ase.calc = calc
    f = scell ase.get_forces()
    forces.append(f)
    print(f"Done displacement {i+1}/{len(supercells)}")

# Build and save force constants
phonon.produce_force_constants(forces=forces)
phonon.symmetrize_force_constants()
write_FORCE_CONSTANTS(phonon.force_constants, filename=f"{name}-FORCE_CONSTANTS")
phonon.save(filename=f"{name}-phonopy.yaml")

# Phonon band structure
paths = {
    "zro2-c": [ [0.5, 0.5, 0.5], [0.0, 0.0, 0.0], [0.0, 0.5, 0.5], [0.25, 0.75, 0.5],
                [0.375, 0.75, 0.375], [0.0, 0.0, 0.0] ],
    "zro2-t": [ [0.0, 0.0, 0.0], [0.0, 0.5, 0.0], [0.5, 0.5, 0.0], [0.0, 0.0, 0.0], [0.0, 0.0, 0.5],
                [0.0, 0.5, 0.5], [0.5, 0.5, 0.5] ],
    "zro2-m": [ [0.0, 0.0, 0.0], [0.0, 0.5, 0.0], [0.0, 0.5, 0.5], [0.0, 0.0, 0.5], [0.0, 0.0, 0.0],
                [-0.5, 0.0, 0.5], [-0.5, 0.5, 0.5], [-0.5, 0.0, 0.0], [0.0, 0.0, 0.0] ] }
path = [paths[name]]
bands, connections = get_band_qpoints_and_path_connections( path, npoints=51 )
nac_params = parse_BORN( phonon.primitive, filename=f"ph-ref/{name}-born", symprec=1e-4)
nac_params["factor"] = Hartree * Bohr
phonon.nac_params = nac_params
phonon.run_band_structure( bands, path_connections=connections )
phonon.write_yaml_band_structure(filename=f"{name}-band.yaml")
print("Done.")
```

The script `phonons.py` reproduced above uses [phonopy](#) to create a $2 \times 2 \times 2$ supercell of the unit cell and generate finite atomic displacements of 0.015 Å. The MACE model is then used through the

ASE calculator in order to evaluate the resulting forces on atoms for each displaced supercell. From the forces, phonopy constructs the real-space interatomic force constants and then calculates phonon dispersions along the specified q -paths. Notice how fast the calculation is using MACE!

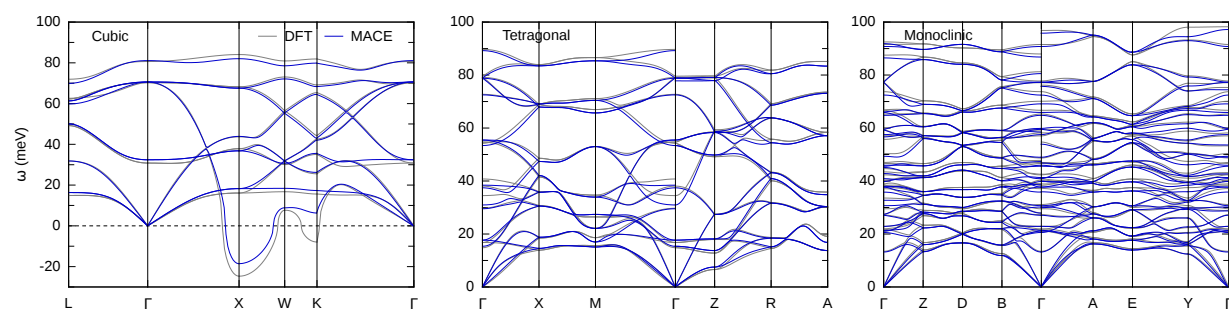
Note: The number of possible displacements is reduced using symmetry. Here we set the symmetry tolerance to $1\text{e-}4$ (slightly larger than the default value of $1\text{e-}5$) since the structures obtained from the structural optimizations have not been symmetrized.

Note: We account for the non-analytical terms in the dynamical matrix to correct for the long-range electrostatics. For this, we need Born effective charges and dielectric constants, provided in the files `ph-ref/name-born`. These files can be produced from the usual QE output, for example, using the `phonopy-qe-born` command.

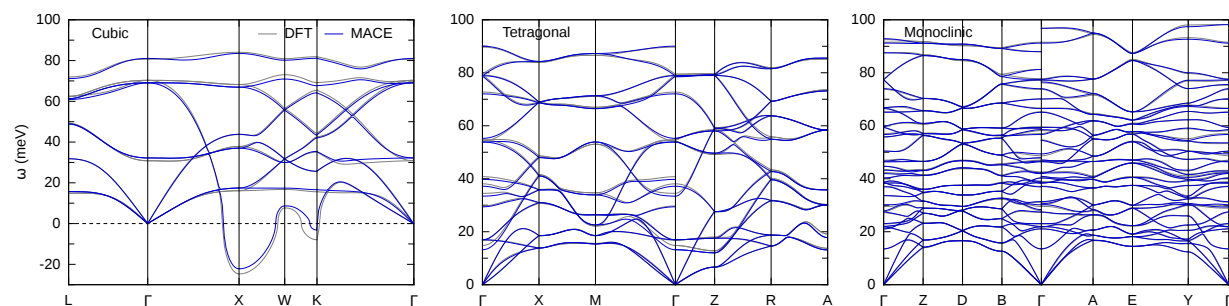
The phonon dispersions are written in a simple text format. We can plot them, together with the reference DFT ones³ in the folder `ph-ref/`, with the gnuplot script `plot-ph.p` provided:

```
$ gnuplot plot-ph.p
$ evince plot-phonons.pdf
```

You should obtain something similar to this:



► Try redoing the exercise with the 'medium' MACE model. To do that, go to this folder and run `run3.sh` and `run4.sh` again; these and the python scripts `relax.py` and `phonopy.py` are already copied there and modified for your convenience, as well as the plotting script `plot-ph.p` (changes include the model name and the paths pointing to the `structures/` and `ph-ref/` folders). Observe how the phonons become more accurate, especially for the (unstable) cubic phase. You should obtain a new figure similar to this:



³From [npj Comput. Mater. 7, 156 \(2021\)](#).

Exercise 2

In this exercise we use the MLIP generated in Exercise 1 to run finite-temperature molecular dynamics (MD) orders of magnitude faster than ab initio MD. We will also use an MD simulation to observe the monoclinic-to-tetragonal phase transition.

2.1 Molecular dynamics

Start by navigating to the directory for this exercise, and creating a symbolic link to the 'medium' MACE model from the previous exercise:

```
$ cd ../exercise2
$ ln -s ../exercise1/medium/zro2_MACE_model_stagetwo_medium.model .
```

► Run molecular dynamics in the NPT ensemble (i.e., a thermostat and a barostat are applied to keep temperature and pressure fixed) at $T = 2000$ K and ambient pressure, starting from a $3 \times 3 \times 3$ supercell of the tetragonal structure:

```
$ sbatch run1.sh
```

```
--
#!/bin/bash
#SBATCH -J myjob      # Job name
#SBATCH -N 1         # Total # of nodes
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=56
#SBATCH -t 00:30:00  # Run time (hh:mm:ss)
#SBATCH -A DMR23048
#SBATCH -p small
#SBATCH --reservation=MATCSSI_Norm_June16

module load python3/3.9.2
source $WORK/MACE/bin/activate
unset PYTHONPATH

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export MKL_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OPENBLAS_NUM_THREADS=$SLURM_CPUS_PER_TASK
export NUMEXPR_NUM_THREADS=$SLURM_CPUS_PER_TASK

sed -i "s/torch\.set_num_threads\.*/torch.set_num_threads(${SLURM_CPUS_PER_TASK})/" md-npt1.py
cp ../exercise1/zro2-t-relax.cif .
python md-npt1.py zro2-t 2000
```

```
#
import sys
import torch
import numpy as np
import matplotlib.pyplot as plt
from ase import units, Atoms
from ase.io import read, write
from ase.md.npt import NPT
from ase.md.velocitydistribution import MaxwellBoltzmannDistribution, Stationary
from mace.calculators import MACECalculator

torch.set_num_threads(28)
name = sys.argv[1]
T_target = float(sys.argv[2]) # temperature (K)
model = "zro2_MACE_model_stagetwo_medium.model"
sc_size = (3,3,3)             # change for a bigger supercell
tot_t = 4                     # total trajectory length (ps)
dt = 2.0                       # timestep (fs)
log_interval = 10              # how often T,E,atoms recorded (steps)
traj_interval = 20            # how often atoms recorded (steps)
```

```

# NPT parameters
P_target = 0.0                # external pressure (eV/Ang^3)
ttime = 25.0 * units.fs      # thermostat time scale
ptime = 75.0 * units.fs      # barostat time scale
B = 1.0                      # tune if needed
pfactor = (ptime**2)*B

atoms = read(f"{name}-relax.cif").repeat(sc_size)
atoms.pbc = True
atoms.calc = MACECalculator( model_paths=model,device="cpu",default_dtype="float32" )

# initialise (NPT)
rng = np.random.default_rng(123)
MaxwellBoltzmannDistribution(atoms, temperature_K=T_target, rng=rng)
Stationary(atoms)
dyn = NPT( atoms, timestep=dt*units.fs, temperature_K=T_target,
           externalstress=P_target, ttime=ttime, pfactor=pfactor )

def atoms_write(atoms):
    return Atoms( symbols=atoms.get_chemical_symbols(),
                  positions=atoms.get_positions(), cell=atoms.cell, pbc=atoms.pbc )

def run_md(tot_t):
    ntot = int(round(tot_t * 1000 / dt))
    data = []
    log_file = f"{name}-T{T_target}.log"
    traj_file = f"{name}-T{T_target}.xyz"
    log = open(log_file, "w")
    log.write("# time(ps) T(K) Epot/atom(eV) V(A^3) P(eV/A^3) a(A) b(A) c(A)\n")

    def write_log():
        time = dyn.get_time() / units.fs / 1000.0
        T = atoms.get_temperature()
        Epot = atoms.get_potential_energy() / len(atoms)
        V = atoms.get_volume()
        stress = atoms.get_stress(include_ideal_gas=True)
        P = -np.mean(stress[:3]) # mean of trace
        a, b, c = atoms.cell.lengths()
        data.append([time, T, Epot, V, P, a, b, c])
        log.write( f"{time:12.4f} {T:12.4f} {Epot:14.8f} {V:12.4f} {P:12.4f}"
                  f"{a:12.4f} {b:12.4f} {c:12.4f}\n" )
        if dyn.nsteps % 200 == 0:
            log.flush()

    def write_traj():
        write(traj_file, atoms_write(atoms), append=True)

    dyn.attach(write_log, interval=log_interval)
    dyn.attach(write_traj, interval=traj_interval)
    dyn.run(ntot)
    log.close()

    write(f"{name}-T{T_target}-final.cif", atoms)
    print("Finished MD")

    return np.array(data)

# Run MD trajectory and plot
data = run_md(tot_t)

# (Plotting part not reproduced here)

```

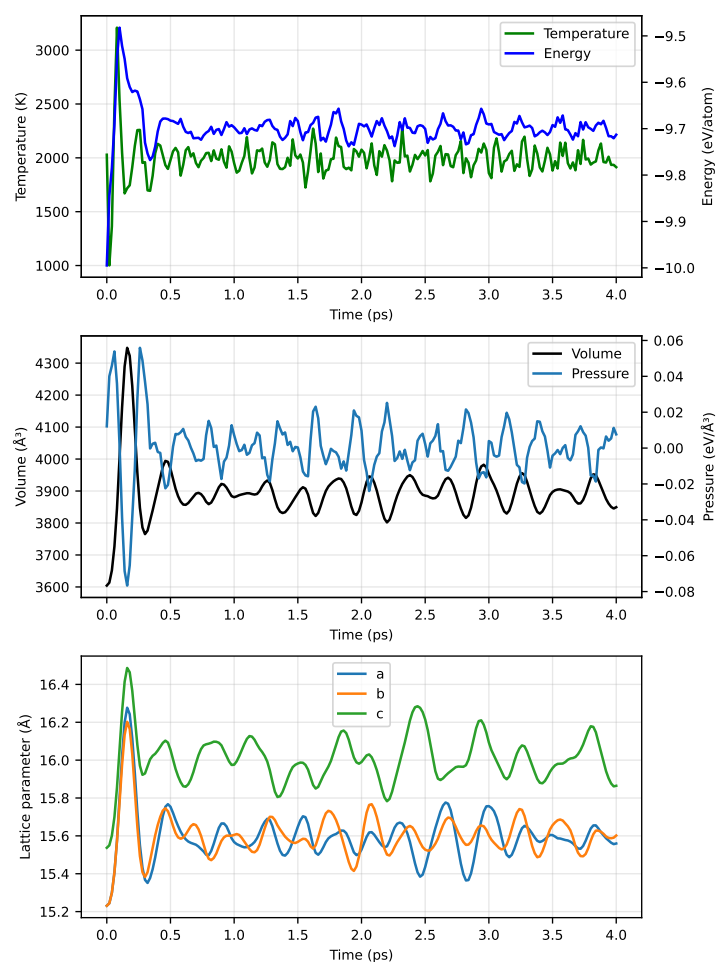
The calculation needs a few minutes to finish. In the meantime, inspect the file `md-npt1.py`. As in Exercise 1, we run calculations using our MACE model via the ASE calculator. The various parameters, including simulation time (`tot_t`, 4 ps) and time step (`dt`, 2 fs) are set at the beginning. The initial velocities are drawn randomly from a Maxwell-Boltzmann distribution, and the dynamics is run in the NPT ensemble. The script runs the MD and monitors temperature, energy, volume and pressure, as well as the lattice parameters, at each step and outputs these data every 10 steps in the

file `zro2-t-T2000.0.log`.

► Analyze the MD trajectory logfile. It is a gnuplot-style file that you can easily plot or use for later analyses. We already have a plot automatically generated by the python script:

```
$ evince zro2-t-T2000.0.pdf
```

It should look similar to the following figure. Does the model remain stable? Is the structure still tetragonal?



Note: the atomic coordinates and lattice are recorded (every set number of steps) in the file `zro2-t-T2000.0.xyz`. You can copy it to your computer and visualize it, for example, by running OVITO (<https://www.ovito.org/#download>; OVITO Basic).

Note: In `md-npt1.py`, `ttime` is the characteristic timescale of the thermostat, and `pfactor` is defined as `ptime**2 * B`, with `ptime` the characteristic barostat timescale and `B` the bulk modulus. As indicated in ASE's documentation, 25 fs for `ttime` and 75 fs for `pftime` are usually good values, but you should monitor the temperature and pressure fluctuations. Too small values may cause instabilities and/or incorrect fluctuations, too large values cause a slow oscillation.

► Calculate the average lattice parameters at this temperature by running the following script from the command line:

```
$ python lattice-par.py zro2-t 2000 2
```

```

# lattice-par.py
import sys
import numpy as np

name = sys.argv[1]
T = float(sys.argv[2])
t_start = float(sys.argv[3]) # discard data before t_start (ps)
log_file = f"{name}-T{T}.log"

data = np.loadtxt(log_file, comments="#")
time = data[:, 0]
a = data[:, 5]
b = data[:, 6]
c = data[:, 7]

mask = time >= t_start
a_ave = np.mean(a[mask]); a_std = np.std(a[mask])
b_ave = np.mean(b[mask]); b_std = np.std(b[mask])
c_ave = np.mean(c[mask]); c_std = np.std(c[mask])

print(f"File: {log_file}")
print(f"Averaging from time >= {t_start} ps")
print(f"a_avg = {a_ave:12.4f} Å   std = {a_std:12.4f} Å")
print(f"b_avg = {b_ave:12.4f} Å   std = {b_std:12.4f} Å")
print(f"c_avg = {c_ave:12.4f} Å   std = {c_std:12.4f} Å")

```

The script takes the average of the lattice parameters after discarding the first 2 ps of the trajectory. How do the lattice parameters compare to the ones obtained from the ground-state relaxation (after dividing them by 3)?

► This was a short MD run; try running a longer simulation and analyze the trajectory.

2.2 Structural phase transition

Now we perform an MD simulation in which we quickly increase the temperature from values below the monoclinic-to-tetragonal transition to values above. Again, we allow the volume and the lattice parameters to adjust to the temperature by fixing the external pressure (NPT ensemble). Then, we keep the temperature at 2000 K and let the system evolve for a few ps.

► Run this MD sequence starting from the monoclinic structure, first going from 1000 K to 2000 K and then fixing the temperature at 2000 K:

```
$ sbatch run2.sh
```

where `run2.sh` is created by replacing the last 3 lines of `run1.sh` with:

```

-- run2.sh
...
...
sed -i "s/torch\.set_num_threads\.*/torch.set_num_threads(${SLURM_CPUS_PER_TASK})/" md-npt2.py
sed -i "s/torch\.set_num_threads\.*/torch.set_num_threads(${SLURM_CPUS_PER_TASK})/" md-npt3.py
cp ../exercise1/zro2-m-relax.cif .
python md-npt2.py zro2-m 1000 2000
python md-npt3.py zro2-m 2000

```

and `md-npt2.py` performs a temperature ramp, with the target T updated every 200 fs:

```

# md-npt2.py
import sys
import torch
import numpy as np

```

```

import matplotlib.pyplot as plt
from ase import units, Atoms
from ase.io import read, write
from ase.md.npt import NPT
from ase.md.velocitydistribution import MaxwellBoltzmannDistribution, Stationary
from mace.calculators import MACECalculator

torch.set_num_threads(56)
name = sys.argv[1]
T_in = float(sys.argv[2])      # initial temperature (K)
T_target = float(sys.argv[3])  # final temperature (K)
model = "zro2_MACE_model_stagetwo.model"
sc_size = (3,3,3)              # change for a bigger supercell
tot_t = 3                      # total trajectory length (ps)
dt = 2.0                       # timestep (fs)
chunk_ps = 0.2                 # interval to update target T (ps)
log_interval = 10              # how often T,E,atoms recorded (steps)
traj_interval = 20            # how often atoms recorded (steps)
# NPT parameters
P_target = 0.0                 # external pressure (eV/Ang^3)
ttime = 25.0 * units.fs        # thermostat time scale
ptime = 75.0 * units.fs        # barostat time scale
B = 1.0                        # tune if needed
pfactor = (ptime**2)*B

atoms = read(f"{name}-relax.cif").repeat(sc_size)
atoms.pbc = True
atoms.calc = MACECalculator( model_paths=model,device="cpu",default_dtype="float32" )

# initialise (NPT)
rng = np.random.default_rng(123)
MaxwellBoltzmannDistribution(atoms, temperature_K=T_in, rng=rng)
Stationary(atoms)

def atoms_write(atoms):
    return Atoms( symbols=atoms.get_chemical_symbols(),
                  positions=atoms.get_positions(), cell=atoms.cell, pbc=atoms.pbc )

def run_md_ramp(T0, T1, tot_t):
    n_chunks = int(round(tot_t/chunk_ps))
    steps_per_chunk = int(round(chunk_ps * 1000 / dt))
    temps = np.linspace(T0, T1, n_chunks)
    data = []
    log_file = f"{name}-ramp-T{T_target}.log"
    traj_file = f"{name}-ramp-T{T_target}.xyz"
    dyn = NPT( atoms, timestep=dt*units.fs, temperature_K=T0,
               externalstress=P_target, ttime=ttime, pfactor=pfactor )

    with open(log_file, "w") as log:
        log.write("# time(ps) T(K) Epot/atom(eV) V(A^3) P(eV/A^3) a(A) b(A) c(A)\n")

    def write_log():
        time = dyn.get_time() / units.fs / 1000
        T = atoms.get_temperature()
        Epot = atoms.get_potential_energy() / len(atoms)
        V = atoms.get_volume()
        stress = atoms.get_stress(include_ideal_gas=True)
        P = -np.mean(stress[:3]) # mean of trace
        a, b, c = atoms.cell.lengths()
        data.append([time, T, Epot, V, P, a, b, c])
        log.write( f"{time:12.4f} {T:12.4f} {Epot:14.8f} {V:12.4f} {P:12.4f}"
                   f"{a:12.4f} {b:12.4f} {c:12.4f}\n" )

    def write_traj():
        write(traj_file, atoms_write(atoms), append=True)

    dyn.attach(write_log, interval=log_interval)
    dyn.attach(write_traj, interval=traj_interval)

    for target_T in temps:
        dyn.set_temperature(temperature_K=float(target_T))

```

```

        dyn.run(steps_per_chunk)

    write(f"{name}-ramp-T{T_target}-final.cif", atoms)
    print("Finished MD")

    return np.array(data)

# Run MD trajectory and plot
data = run_md_ramp(T_in, T_target, tot_t)

# (Plotting part not reproduced here)

```

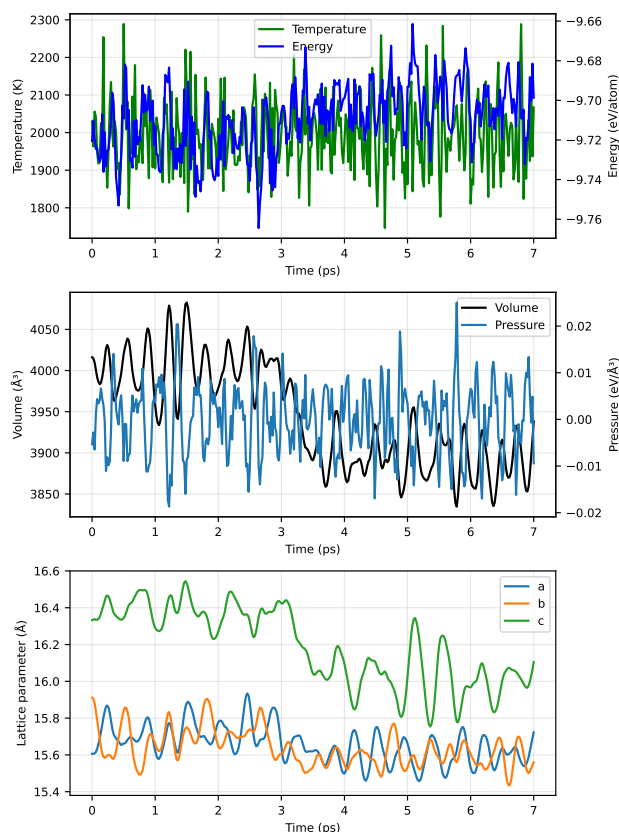
while `md-npt3.py` is the same as `md-npt1.py`, but it simulates 7 ps and reads in the structure obtained from the previous MD run (and hence it does not replicate the unit cell). This calculation will take over 10 minutes, so you are welcome to start looking at the next exercise in the meantime.

► Inspect the MD trajectories:

```
$ evince zro2-m-ramp-T2000.0.pdf
```

```
$ evince zro2-m-T2000.0.pdf
```

The second one should look similar to the figure below. If you are lucky, you will see that the volume abruptly decreases (if not, you may have to run the simulation a little longer): this is an indication of the (first-order) transition to the tetragonal phase. In practice, this is a very crude calculation, and instead you should heat the system more slowly or perform longer MD simulations at temperatures closer to the transition temperature. Due to time constraints, here we used a short MD at T substantially higher than the transition temperature of about 1400 K.



Exercise 3

In this exercise we will show how to train a MACE potential for silica (SiO_2), assess its accuracy, and use it to compute vibrational densities of states for representative vitreous structures.

3.1 MACE training for SiO_2

Navigate to the directory for this exercise:

```
$ cd ../exercise3; ls
```

In the subfolder `database/` you can find the file `sio2-train-all.xyz` containing the training dataset we will use. This is a large dataset of 3074 structures including crystalline, liquid and amorphous structures of SiO_2 calculated using the SCAN functional [see [npj Comput. Mater.](#) **8**, 90 (2022)].

You can train the potential by submitting the job script `run1.sh`, however, this is going to take too long for the tutorial. Instead, we provide the model already trained in the folder:

```
ls sio2_MACE_model_stagetwo.model
```

► Inspect the MACE training parameters in `run1.sh`:

```
--                                                                 run1.sh
#!/bin/bash
#SBATCH -J myjob           # Job name
#SBATCH -N 1              # Total # of nodes
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=28
#SBATCH -t 02:00:00      # Run time (hh:mm:ss)
#SBATCH -A DMR23048
#SBATCH -p small
#SBATCH --reservation=MATCSSI_Norm_June16

module load python3/3.9.2
source $WORK/MACE/bin/activate
unset PYTHONPATH

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export MKL_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OPENBLAS_NUM_THREADS=$SLURM_CPUS_PER_TASK
export NUMEXPR_NUM_THREADS=$SLURM_CPUS_PER_TASK

mace_run_train \
  --name="sio2_MACE_model" \
  --train_file="database/sio2-train-all.xyz" \
  --valid_fraction=0.1 \
  --energy_key="energy" \
  --forces_key="forces" \
  --stress_key="stress" \
  --compute_stress=True \
  --loss="huber" \
  --E0s="{14: -4.06855316, 8: -0.66320916}" \
  --model="ScaleShiftMACE" \
  --num_interactions=2 \
  --num_channels=64 \
  --correlation=3 \
  --max_L=0 \
  --r_max=4 \
  --batch_size=10 \
  --max_num_epochs=50 \
  --ema \
  --ema_decay=0.99 \
  --amsgrad \
  --default_dtype="float32" \
  --device=cpu \
```

```
--seed=123 \  
--swa
```

► Inspect the model parameters, loss, training and validation errors in the reference logfile provided: `ref-logs/sio2_MACE_model_run-123.log`. Why do you think the RMSEs are larger than in Exercise 1?

3.2 Phonons of amorphous structures

For amorphous structures, calculating the vibrational density of states from first principles is very demanding, as symmetries cannot be exploited to reduce the cost of density functional perturbation theory or finite-displacement calculations for phonons as in ordered crystals. Moreover, large super-cells may be required to describe realistic amorphous structures. Fortunately, MLIPs can help.

► Compute the vibrational density of states of representative structures of vitreous SiO_2 . There are three structures labelled `name_sio2.cif`, with `name = model_I, model_II, model_III`, in the folder `structures/` in the Exercise 3 directory, taken from [Phys. Rev. B 79, 064202 \(2009\)](#) (they can be downloaded from [Materials Cloud](#)). Model I contains 144 atoms, while model II and model III contain 72 atoms.

```
$ sbatch run2.sh
```

```
--                                                                 run2.sh  
#!/bin/bash  
#SBATCH -J myjob          # Job name  
#SBATCH -N 1              # Total # of nodes  
#SBATCH --ntasks=1  
#SBATCH --cpus-per-task=8  
#SBATCH -t 00:30:00      # Run time (hh:mm:ss)  
#SBATCH -A DMR23048  
#SBATCH -p small  
#SBATCH --reservation=MATCSSI_Norm_June16  
  
module load python3/3.9.2  
source $WORK/MACE/bin/activate  
unset PYTHONPATH  
  
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK  
export MKL_NUM_THREADS=$SLURM_CPUS_PER_TASK  
export OPENBLAS_NUM_THREADS=$SLURM_CPUS_PER_TASK  
export NUMEXPR_NUM_THREADS=$SLURM_CPUS_PER_TASK  
  
sed -i "s/torch\.set_num_threads\.*/torch.set_num_threads(${SLURM_CPUS_PER_TASK})/" phonons.py  
for name in model_I model_II model_III; do  
    python phonons.py $name  
done
```

```
#                                                                 phonopy.py  
import sys  
import torch  
import numpy as np  
from ase import Atoms  
from ase.io import read  
from ase.optimize import BFGS  
from mace.calculators import MACECalculator  
from phonopy import Phonopy  
from phonopy.structure.atoms import PhonopyAtoms  
from phonopy.file_IO import write_FORCE_CONSTANTS, parse_BORN  
from phonopy.phonon.band_structure import get_band_qpoints_and_path_connections  
from phonopy.units import Hartree, Bohr  
  
torch.set_num_threads(8)
```

```

name = sys.argv[1]
atoms = read(f"structures/{name}_sio2.cif")
sc_size = [1, 1, 1]
calc = MACECalculator( model_paths="sio2_MACE_model_stagetwo.model",
    device="cpu", default_dtype="float32" )
atoms.calc = calc

# Relax structure
opt = BFGS(atoms, trajectory=f"{name}-relax.traj", logfile=f"{name}-relax.log")
opt.run(fmax=0.01)

unitcell = PhonopyAtoms( symbols=atoms.get_chemical_symbols(),
    cell=atoms.cell.array, scaled_positions=atoms.get_scaled_positions() )
# Choose supercell size
phonon = Phonopy( unitcell, supercell_matrix=np.diag(sc_size), primitive_matrix="auto", symprec=1e-4)
# Generate finite displacements (in A)
phonon.generate_displacements(distance=0.015)
supercells = phonon.supercells_with_displacements
print(f"Number of displaced supercells: {len(supercells)}")

forces = []
for i, scell in enumerate(supercells):
    scell ase = Atoms( symbols=scell.symbols,
        cell=scell.cell, scaled_positions=scell.scaled_positions, pbc=True )
    scell ase.calc = calc
    f = scell ase.get_forces()
    forces.append(f)
    print(f"Done displacement {i+1}/{len(supercells)}")

# Build and save force constants
phonon.produce_force_constants(forces=forces)
phonon.symmetrize_force_constants()
write_FORCE_CONSTANTS(phonon.force_constants, filename=f"{name}-FORCE_CONSTANTS")
phonon.save(filename=f"{name}-phonopy.yaml")

# Calculate ph DOS
mesh = [6,6,6]
s = 0.5 # Gaussian broadening (THz)
phonon.run_mesh(mesh)
phonon.run_total_dos(sigma=s)
phonon.write_total_dos(filename=f"{name}-total_dos.dat")
print("Done.")

```

The job script `run2.sh` uses [MACE](#), [ASE](#) and [phonopy](#) to calculate the phonons for all three structures using finite displacements, in a similar way as in Exercise 1. Here, the initial structural relaxation step is included directly in the `phonons.py` script. The final step is to compute the vibrational density of states, which is written to the output files `name-total_dos.dat`.

Note: From the job output, you can check out how many calculations are performed for each model structure.

Note: In this exercise, we are keeping to harmonic phonons. In practice, MLIPs can be very useful for accelerating anharmonic phonon calculations at finite temperature, using methods such as self-consistent phonons or molecular dynamics.

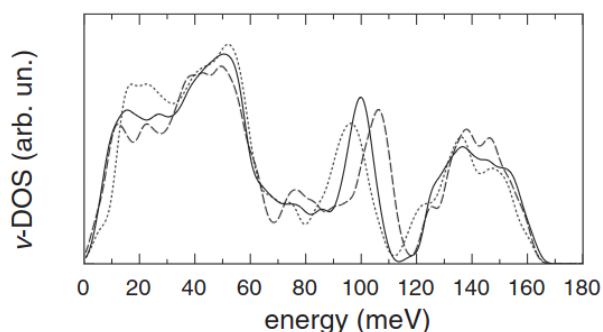
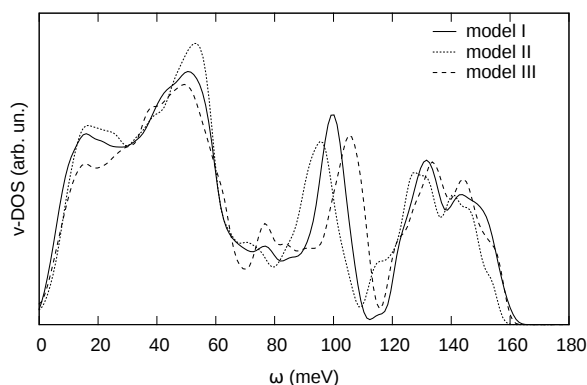
► Use the gnuplot script `plot-dos.p` to plot the density of states for all three models:

```

$ gnuplot plot-dos.p
$ evince plot-dos.pdf

```

You should obtain the following plot (left panel). Compare it with the plot in the right panel, showing DFT calculations from [Phys. Rev. B 79, 064202 \(2009\)](#) (the legend is the same).



Exercise 4

In this exercise we will perform and analyze melt-quench MD simulations to generate vitreous silica configurations using the MACE model from Exercise 3. In this way, we are able to generate our own representative vitreous structures and compare them with the first-principles ones used in the previous exercise.

4.1 Melt-quench molecular dynamics

Start by navigating to the directory for this exercise, and creating a symbolic link to the MACE model from the previous exercise. Also copy the model I reference structure.

```
$ cd ../exercise4
$ ln -s ../exercise3/sio2_MACE_model_stagetwo.model .
$ cp ../exercise3/structures/model_I_sio2.cif .
```

In this exercise we run MD simulations in the NVT ensemble, starting from the model I structure from Exercise 3. We keep the cell parameters fixed in order to reproduce the experimental density of 2.2 g/cm^3 (as in the ab initio calculations performed to obtain the model I structure).

► Run a melt-quench MD simulation to obtain vitreous silica:

```
$ sbatch run1.sh
```

```
--
#!/bin/bash
#SBATCH -J myjob      # Job name
#SBATCH -N 1          # Total # of nodes
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=28
#SBATCH -t 00:30:00  # Run time (hh:mm:ss)
#SBATCH -A DMR23048
#SBATCH -p small
#SBATCH --reservation=MATCSSI_Norm_June16

module load python3/3.9.2
source $WORK/MACE/bin/activate
unset PYTHONPATH

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
export MKL_NUM_THREADS=$SLURM_CPUS_PER_TASK
export OPENBLAS_NUM_THREADS=$SLURM_CPUS_PER_TASK
export NUMEXPR_NUM_THREADS=$SLURM_CPUS_PER_TASK
```

```
sed -i "s/torch\.set_num_threads\.*/torch.set_num_threads(${SLURM_CPUS_PER_TASK})/" md-nvt.py
python md-nvt.py sio2
```

```
# md-nvt.py
import sys
import torch
import numpy as np
import matplotlib.pyplot as plt
from ase import units, Atoms
from ase.io import read, write
from ase.constraints import FixCom
from ase.md.langevin import Langevin
from ase.md.velocitydistribution import MaxwellBoltzmannDistribution, Stationary
from mace.calculators import MACECalculator

torch.set_num_threads(28)
name = sys.argv[1]
model = "sio2_MACE_model_stagetwo.model"
sc_size = (1,1,1) # change for a bigger supercell
dt = 2.0 # timestep (fs)
chunk_ps = 0.2 # interval to update target T (ps)
log_interval = 10 # how often T,E,atoms recorded (steps)
traj_interval = 100 # how often atoms recorded (steps)
friction = 1.0 / (100.0 * units.fs) # Langevin friction
phases = [ ("heat", 300, 3500, 5),
            ("melt", 3500, 3500, 5),
            ("quench", 3500, 300, 10),
            ("equil", 300, 300, 5) ]

atoms = read(f"model_I_sio2.cif").repeat(sc_size)
atoms.pbc = True
atoms.set_constraint(FixCom()) # remove center of mass drift if any
atoms.calc = MACECalculator( model_paths=model,device="cpu",default_dtype="float32" )

#initialise
rng = np.random.default_rng(123)
MaxwellBoltzmannDistribution(atoms, temperature_K=300, rng=rng)
Stationary(atoms)
all_data = {}

def atoms_write(atoms):
    return Atoms( symbols=atoms.get_chemical_symbols(),
                  positions=atoms.get_positions(), cell=atoms.cell, pbc=atoms.pbc )

def run_phase(label, T0, T1, tot_t):
    n_chunks = int(round(tot_t/chunk_ps))
    steps_per_chunk = int(round(chunk_ps * 1000 / dt))
    temps = np.linspace(T0, T1, n_chunks)
    data = []
    log_file = f"{name}-{label}.log"
    traj_file = f"{name}-{label}.xyz"
    dyn = Langevin( atoms, timestep=dt*units.fs, temperature_K=T0, friction=friction, fixcm=False )

    with open(log_file, "w") as log:
        log.write("# time(ps) T(K) Epot/atom (eV)\n")

        def write_log():
            time = dyn.get_time()/units.fs/1000
            T = atoms.get_temperature()
            Epot = atoms.get_potential_energy() / len(atoms)
            data.append([time, T, Epot])
            log.write(f"{time:12.6f} {T:12.4f} {Epot:20.10f} \n")

        def write_traj():
            write(traj_file, atoms_write(atoms), append=True)

        dyn.attach(write_log, interval=log_interval)
        dyn.attach(write_traj, interval=traj_interval)

    for target_T in temps:
```

```

        dyn.set_temperature(temperature_K=float(target_T))
        dyn.run(steps_per_chunk)

    all_data[label] = np.array(data)
    write(f"{name}-{label}-final.cif", atoms)
    print(f"Finished {label}")

# Run MD trajectories
for phase in phases:
    label, T0, T1, tot_t = phase
    run_phase(label, T0, T1, tot_t)

# (Plotting part not reproduced here)

```

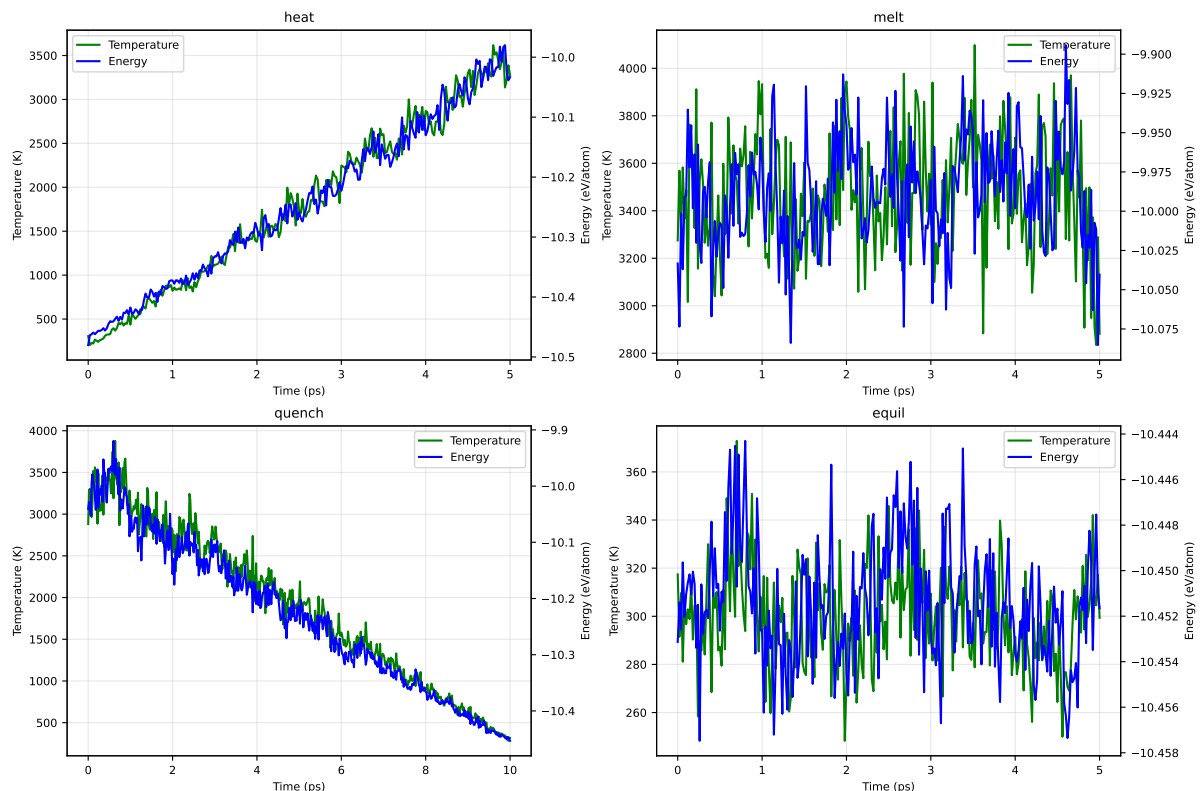
While the simulation runs, inspect the python script `md-nvt.py`. The melt-quench procedure is divided into four steps:

- Heat the structure from 300 K to 3500 K, well above the experimental melting point of 2000 K, over 5 ps.
- Equilibrate the liquid at 3500 K for 5 ps to ensure complete melting.
- More slowly quench the system back to 300 K over 10 ps.
- Equilibrate the structure at 300 K for 5 ps to obtain a vitreous silica configuration.

Note: For this tutorial, the simulation cell and trajectory length are kept small so that the calculation can run quickly. Larger cells and longer simulation times should be used for high-quality results.

The python script produces a logfile and a trajectory file and also automatically gives us a plot, similar to the following one:

\$ `evince sio2-melt-quench.pdf`



► Further analyze the results by calculating the partial pair correlation functions for the liquid and vitreous structures:

```
$ python rdf.py sio2-melt.xyz melt
$ python rdf.py sio2-equil.xyz equil
```

```
#                                                                 rdf.py
import sys
import numpy as np
import matplotlib.pyplot as plt
from ase.io import read
from ase.geometry.rdf import get_rdf

traj_file = sys.argv[1]
name = sys.argv[2]
rmax = 8; nbins = 400
frames = read(traj_file, ":")
pairs = {
    "Si-Si": (14, 14),
    "Si-O": (14, 8),
    "O-O": (8, 8),
}

# Calculate average rdf over the trajectory
rdf_sum = {}
for label in pairs:
    rdf_sum[label] = np.zeros(nbins)

for atoms in frames:
    for label, pair in pairs.items():
        atoms_s = atoms.repeat((2,2,2))
        rdf, r = get_rdf(atoms_s, rmax=rmax, nbins=nbins, elements=pair)
        rdf_sum[label] += rdf

nframes = len(frames)
rdf_avg = {}
for label in pairs:
    rdf_avg[label] = rdf_sum[label] / nframes

# Save data
with open(f"rdf-{name}.dat", "w") as f:
    f.write("# r[Å] g_SiSi g_SiO g_OO\n")
    for i in range(nbins):
        f.write(f"{r[i]:12.6f} "
                f"{rdf_avg['Si-Si'][i]:15.8f} "
                f"{rdf_avg['Si-O'][i]:15.8f} "
                f"{rdf_avg['O-O'][i]:15.8f}\n" )

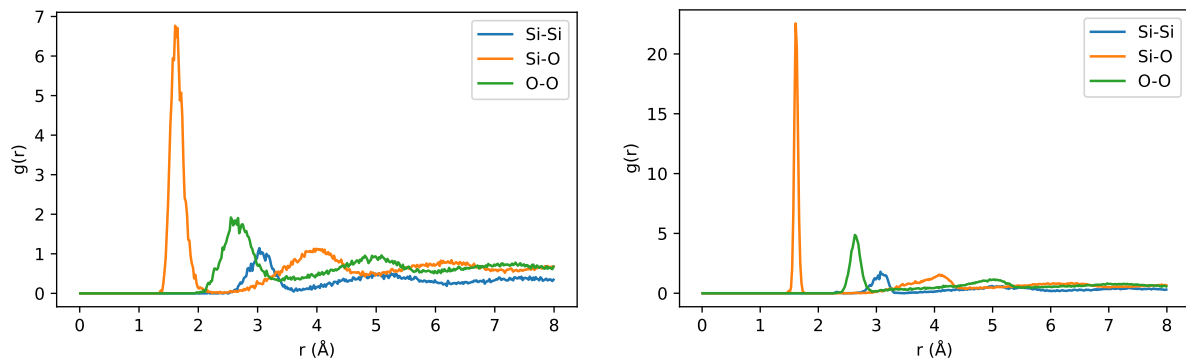
# Plot
plt.figure(figsize=(5,3.2))
for label, rdf in rdf_avg.items():
    plt.plot(r, rdf, label=label)
plt.xlabel("r (Å)")
plt.ylabel("g(r)")
plt.legend()
plt.tight_layout()
plt.savefig(f"rdf-{name}.pdf")
plt.close()

print("Done")
```

► Inspect the plots obtained (an example is shown on the next page). What are the main differences?

```
$ evince rdf-melt.pdf rdf-equil.pdf
```

► Try recalculating the vibrational density of states for the amorphous structure generated by the melt-quench MD using the MACE potential. You can use the `phonopy.py` script from Exercise 3.



How does the density of states compare with the previous results?

► Try performing the melt-quench procedure again with a larger structure, e.g., starting from a $2 \times 2 \times 2$ supercell of the initial one, and recalculate the pair correlation functions.

Appendix 1

```
# Script for plotting MACE vs. DFT data for train and test datasets
import numpy as np
import matplotlib.pyplot as plt
from ase.io import read

datasets = {
    "Train": ("database/zro2-train.xyz", "train_mace_eval.xyz"),
    "Test": ("database/zro2-test.xyz", "test_mace_eval.xyz"),
}

properties = {
    "Energy": {
        "ref_key": "energy",
        "pred_key": "MACE_energy",
        "kind": "energy",
        "xlabel": "DFT energy (eV/atom)",
        "ylabel": "MACE energy (eV/atom)",
    },
    "Forces": {
        "ref_key": "forces",
        "pred_key": "MACE_forces",
        "kind": "forces",
        "xlabel": "DFT force (eV/Å)",
        "ylabel": "MACE force (eV/Å)",
    },
    "Stress": {
        "ref_key": "stress",
        "pred_key": "MACE_stress",
        "kind": "stress",
        "xlabel": "DFT stress (eV/Å³)",
        "ylabel": "MACE stress (eV/Å³)",
    },
}

def get_property(atoms, key):
    if key in atoms.info:
        return atoms.info[key]
    if key in atoms.arrays:
        return atoms.arrays[key]
    if atoms.calc is not None and key in atoms.calc.results:
```

plot-parity.py

```

        return atoms.calc.results[key]

def get_stress(s):
    s = np.asarray(s)
    if s.shape == (3, 3):
        return np.array([s[0, 0], s[1, 1], s[2, 2], s[1, 2], s[0, 2], s[0, 1]])
    return s.ravel()

def get_values(atoms, key, kind):
    value = get_property(atoms, key)
    if kind == "energy":
        return np.array([value/len(atoms)])
    if kind == "forces":
        return np.asarray(value).ravel()
    if kind == "stress":
        return get_stress(value)

def collect_data(ref_file, pred_file, prop):
    ref = read(ref_file, ":")
    pred = read(pred_file, ":")
    x, y = [], []
    for a_ref, a_pred in zip(ref, pred):
        x.append(get_values(a_ref, prop["ref_key"], prop["kind"]))
        y.append(get_values(a_pred, prop["pred_key"], prop["kind"]))
    return np.concatenate(x), np.concatenate(y)

def parity_plot(ax, x, y, title, xlabel, ylabel):
    amin = min(x.min(), y.min())
    amax = max(x.max(), y.max())
    pad = 0.05 * (amax - amin)
    ax.scatter(x, y, s=10, alpha=0.6)
    ax.plot([amin - pad, amax + pad], [amin - pad, amax + pad], "k--", lw=1)
    ax.set_xlim(amin - pad, amax + pad)
    ax.set_ylim(amin - pad, amax + pad)
    ax.set_aspect("equal", adjustable="box")
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.set_title(f"{title}")

fig, axes = plt.subplots(2,3,figsize=(11.6,7),dpi=100)

for row, (dataset_name, files) in enumerate(datasets.items()):
    ref_file, pred_file = files
    for col, (prop_name, prop) in enumerate(properties.items()):
        x, y = collect_data(ref_file, pred_file, prop)
        parity_plot( axes[row,col], x, y, title=f"{dataset_name} {prop_name}",
                    xlabel=prop["xlabel"], ylabel=prop["ylabel"] )

plt.tight_layout()
plt.savefig("plot-parity.png", dpi=300)
plt.close()

```
